Sliding Window Revision

10 problems

Fast recall cards for the current sliding window package. Picture the window boundary movement, then check the sample and bug magnets before opening the Java file. Exported in expanded card mode.

#01 MaxSumSubarrayOfSize

Easy

Fixed-size window sum Easy Source

Before reading: what changes when a fixed-size window moves one step right?

PROBLEM DESCRIPTION

Given an array and a fixed window size k, return the maximum sum of any contiguous subarray of exactly k elements.

INPUT

`arr = [2, 3, 4, 1, 5], k = 2`

OUTPUT

7

TIME

O(n)

SPACE

O(1)

DATA STRUCTURES

Running sum, left index, right index

WHEN TO RECOGNIZE IT

When every candidate subarray has the same fixed length and only one element changes as the window slides.

VISUAL HOOK

2 3 | 4 1 5

remove left add right

3 + 4 = 7 best so far

1. Sum the first k values once.
2. Slide by removing the outgoing left value and adding the incoming right value.
3. Keep the maximum sum seen after each slide.

Think of a frame of size k moving one step at a time. Each move drops the left value and picks up the next right value, so the sum updates without recounting the whole frame.

CORE MOVE

Build the first window sum. Then slide by subtracting arr[left] and adding arr[right]. Update the best sum after every slide.

CODE SKELETON

```
sum = first k values
best = sum
left = 0; right = k
while right < n:
  sum = sum - arr[left] + arr[right]
  best = max(best, sum)
  left++
  right++
return best
```

BRUTE FORCE / BASELINE

Try every window and sum its k elements from scratch. It is simple but costs O(n * k).

ALTERNATE APPROACHES

- A prefix sum array can answer each fixed window sum in O(1), but uses O(n) extra space.
- The running-window sum is preferred here because it keeps O(1) space.

BUG MAGNETS

- Initialize the first k values before sliding.
- After each slide, move both boundaries exactly one step.
- This fixed-size pattern is not a shrinking/expanding variable window.

#02 MaximumAverageSubarrayOfSize

Easy

Fixed-size window average Easy Source LeetCode

Before reading: why can maximum average be found by carrying only the window sum?

PROBLEM DESCRIPTION

Given an integer array and a fixed window size k, return the maximum average value of any contiguous subarray of exactly k elements.

INPUT

`nums = [1, 12, -5, -6, 50, 3], k = 4`

OUTPUT

`12.75`

TIME

O(n)

SPACE

O(1)

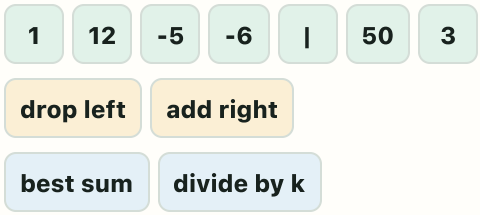
DATA STRUCTURES

Running sum, left index, right index

WHEN TO RECOGNIZE IT

When every candidate has the same length k and the average is just the window sum divided by that fixed k.

VISUAL HOOK



1. Sum the first k values and compute the first average.
2. Slide by subtracting the outgoing left value and adding the incoming right value.
3. Compare the new average, making sure the division is double division.

Do not recalculate averages from scratch. Carry the window sum forward; since k never changes, the window with the best sum also has the best average.

CORE MOVE

Build the first k-length sum, store its average or best sum, then slide by subtracting nums[left] and adding nums[right]. Compare each new average.

CODE SKELETON

```
sum = first k values
bestAvg = (double) sum / k
left = 0; right = k
while right < n:
  sum = sum - nums[left] + nums[right]
  bestAvg = max(bestAvg, (double) sum / k)
  left++
  right++
return bestAvg
```

BRUTE FORCE / BASELINE

For every starting index, sum the next k values and compute the average. It is correct but costs O(n * k).

ALTERNATE APPROACHES

- Track maxSum instead of maxAvg and divide once at the end; this avoids repeated division.
- Prefix sums can compute each window sum quickly but use O(n) extra space.

BUG MAGNETS

- Cast to double before division, otherwise integer division loses the decimal.
- Negative numbers are allowed, so do not initialize the best answer to zero blindly.
- Because k is fixed, this is a slide-only window, not a shrink-on-condition window.

#03 MinimumSizeSubarraySum

Medium

Expand then shrink Medium Source LeetCode

Before reading: when the sum reaches target, why do you keep shrinking instead of stopping?

PROBLEM DESCRIPTION

Given a target and an array of positive integers, return the minimum length of a contiguous subarray whose sum is at least target. Return 0 if no such subarray exists.

INPUT

```
target = 7, nums = [2, 3, 1, 2, 4, 3]
```

OUTPUT

```
2
```

TIME

O(n)

SPACE

O(1)

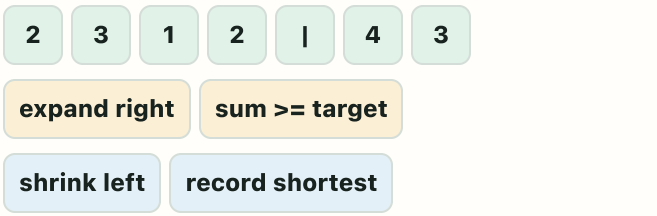
DATA STRUCTURES

Variable window, running sum, left index, right index

WHEN TO RECOGNIZE IT

When all numbers are positive and you need the shortest subarray that reaches at least a target sum.

VISUAL HOOK



1. Expand right and add each value to the running sum.
2. Once sum is at least target, record the current length.
3. Keep removing from the left while the window is still valid to find the shortest version.

Picture the window as a belt that grows until it is strong enough, then tightens from the left as much as possible while still meeting the target.

CORE MOVE

Add nums[right] as the window expands. While sum is at least target, update the best length, subtract nums[left], and move left to try to make the window shorter.

CODE SKELETON

```
left = 0; sum = 0; best = infinity
for right in 0..n-1:
  sum += nums[right]
  while sum >= target:
    best = min(best, right - left + 1)
    sum -= nums[left]
    left++
return best == infinity ? 0 : best
```

BRUTE FORCE / BASELINE

Try every start index and extend right until the sum reaches target. It is clear but can take O(n^2).

ALTERNATE APPROACHES

- Prefix sums plus binary search can work in O(n log n), but it is more machinery than needed for positive numbers.
- The O(n) shrinking window relies on every number being positive; with negatives, shrinking no longer behaves predictably.

BUG MAGNETS

- Use while sum >= target, not if, because one right move may allow multiple left shrinks.
- Update the answer before subtracting nums[left].
- Return 0 when no valid window was ever found.

#04 LongestSubstringWithKMostDistinctCharacters

Medium

At-most-K distinct window Medium Source LeetCode

Before reading: when does removing from the left actually reduce distinct count?

PROBLEM DESCRIPTION

Given a string and an integer k, return the length of the longest contiguous substring that contains at most k distinct characters.

INPUT

`s = "aabacbebebe", k = 3`

OUTPUT

7

TIME

O(n)

SPACE

O(k)

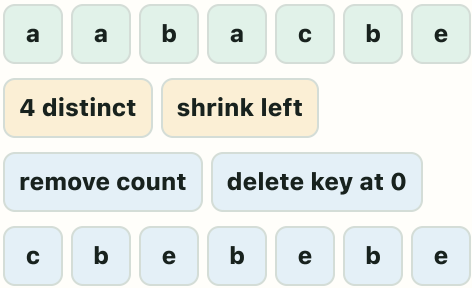
DATA STRUCTURES

HashMap<Character, Integer>, left index, right index

WHEN TO RECOGNIZE IT

When the window is valid while the number of distinct keys is at most k, and repeated characters should not force shrinking.

VISUAL HOOK



1. Expand right and increment the character count in the map.
2. If the map has more than k keys, shrink from the left.
3. Only remove a map key when that character count becomes zero.

Think of the map as a bank of character counts. The distinct count is the number of keys, and a key leaves the bank only when its count becomes zero.

CORE MOVE

Expand right and increment that character count. If the map has more than k keys, shrink left until enough character counts hit zero and are removed. Track the longest valid window.

CODE SKELETON

```
left = 0; best = 0; count = empty map
for right in 0..n-1:
  count[s[right]]++
  while count.size() > k:
    count[s[left]]--
    if count[s[left]] == 0: remove s[left]
    left++
  best = max(best, right - left + 1)
return best
```

BRUTE FORCE / BASELINE

Check every substring and count distinct characters each time. It is straightforward but can become O(n^2) or worse.

ALTERNATE APPROACHES

- For a small fixed alphabet, an int array can replace the map, but the shrink rule is the same.
- A last-seen-index approach can jump the left pointer, but the frequency map version is easier to generalize.

BUG MAGNETS

- At most k means fewer than k distinct characters is still valid.
- Shrink while map.size() > k, not just once.
- Remove a character key only when its frequency becomes zero.

#05 FruitsInBasket

Medium

At-most-2 types window Medium Source LeetCode

Before reading: when a third fruit type appears, which basket type actually leaves?

PROBLEM DESCRIPTION

Given a row of fruit trees where each number is a fruit type, return the longest contiguous stretch you can collect when you only have two baskets, and each basket can hold one fruit type.

INPUT

```
fruits = [1, 2, 1, 3, 3, 2, 2, 2, 4, 2]
```

OUTPUT

```
5
```

TIME

O(n)

SPACE

O(1)

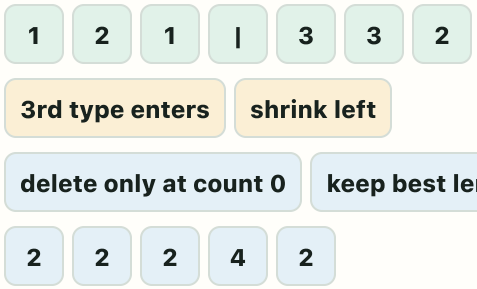
DATA STRUCTURES

HashMap<Integer, Integer>, left index, right index

WHEN TO RECOGNIZE IT

When the window is valid as long as it contains at most two distinct values, and repeated values can keep extending the same window.

VISUAL HOOK



1. Expand right and increment the count for the current fruit type.
2. If there are more than two fruit types, shrink from the left.
3. A fruit type leaves only when its count drops to zero; then the window is valid again.

Picture two baskets as two map keys. The window can grow while only two keys are present; when a third type enters, move left until one old fruit type count becomes zero and leaves.

CORE MOVE

Expand right and count that fruit type. If more than two types are inside, shrink from the left until the map has only two keys again. Track the longest valid stretch after each right move.

CODE SKELETON

```
left = 0; best = 0; count = empty map
for right in 0..n-1:
  count[fruits[right]]++
  while count.size() > 2:
    count[fruits[left]]--
    if count[fruits[left]] == 0: remove
fruits[left]
    left++
  best = max(best, right - left + 1)
return best
```

BRUTE FORCE / BASELINE

Try every starting tree and walk right until a third fruit type appears. It is easy to reason about but can take O(n^2).

ALTERNATE APPROACHES

- A last-seen position approach can jump left past the older fruit type, but the count map is easier to reuse for at-most-K problems.
- Because only two baskets are allowed, the extra space is O(1) even though a map is used.

BUG MAGNETS

- This is exactly at most two distinct types, so one type is also valid.
- Shrink while map.size() > 2, because one removal may not delete a fruit type.
- Remove a fruit type from the map only when its count becomes zero.

#06 LongestSubstringWithoutRepeatingChar

Medium

No-repeat window Medium Source LeetCode

Before reading: why does a duplicate force the left pointer to move until the old copy is removed?

PROBLEM DESCRIPTION

Given a string, return the length of the longest contiguous substring where no character appears more than once.

INPUT

`s = "pwwkew"`

OUTPUT

3

TIME

O(n)

SPACE

O(k), where k is the character set size

DATA STRUCTURES

HashSet<Character>, left index, right index

WHEN TO RECOGNIZE IT

When the current window is valid only if every character inside it is unique.

VISUAL HOOK

1. Use a set to represent the characters currently inside the window.
2. When s[right] is already in the set, remove from the left until that duplicate disappears.
3. Add s[right], then record the current valid window length.

Think of the set as the current clean window. A duplicate cannot enter until the old copy has been removed from the left side.

CORE MOVE

Move right across the string. If the incoming character is already in the set, keep removing left characters until that character is gone. Then add it and update the best window length.

CODE SKELETON

```
left = 0; best = 0; seen = empty set
for right in 0..n-1:
  while seen contains s[right]:
    seen.remove(s[left])
    left++
  seen.add(s[right])
  best = max(best, right - left + 1)
return best
```

BRUTE FORCE / BASELINE

Try every substring and check whether all characters are unique. It is correct but costs O(n^2) or worse depending on the uniqueness check.

ALTERNATE APPROACHES

- A HashMap of last seen indexes can jump left directly past the previous duplicate, which is also O(n).
- For ASCII-only input, a fixed array can replace the set or map.

BUG MAGNETS

- Shrink while the duplicate still exists, not just one step.
- Update the answer after the window is valid again.
- This asks for substring length, so the window must remain contiguous.

#07 LongestRepeatingCharacterSubString

Medium

Replace minority chars Medium Source LeetCode

Before reading: in this window, how many characters are not the majority character?

PROBLEM DESCRIPTION

Given an uppercase string and k replacements, return the longest contiguous substring that can be turned into one repeated character by changing at most k characters.

INPUT

`s = "AABABBAAC", k = 2`

OUTPUT

5

TIME

O(26 * n)

SPACE

O(1)

DATA STRUCTURES

int[26] frequency array, left index, right index

WHEN TO RECOGNIZE IT

When a window is valid if all non-majority characters inside it can be replaced within k moves.

VISUAL HOOK

A A B A B

len 5 max A = 3

5 - 3 = 2 replacements valid

A A A A A

1. Count uppercase letters inside the current window.
2. Use the most frequent letter as the target character.
3. If window length minus that max frequency is more than k, shrink left until the replacement cost fits.

Picture choosing the most common letter in the window as the target. Every other character is a replacement cost, so the window is valid when window length minus max frequency is at most k.

CORE MOVE

Expand right and count the new letter. Find the largest count inside the window. If window length minus that max count is more than k, shrink from the left until the window can be made uniform again.

CODE SKELETON

```
left = 0; best = 0; freq[26] = 0
for right in 0..n-1:
  freq[s[right]]++
  while windowLength - max(freq) > k:
    freq[s[left]]--
    left++
  best = max(best, right - left + 1)
return best
```

BRUTE FORCE / BASELINE

Try every substring and count how many changes are needed to make it all A, all B, and so on. It works but is too slow for long strings.

ALTERNATE APPROACHES

- Track a non-decreasing max frequency while sliding to make the solution O(n); it is faster but slightly trickier to reason about.
- Because the alphabet is only uppercase English letters, the frequency array is simpler than a map.

BUG MAGNETS

- You do not replace every A or every B in the whole string, only characters inside the chosen window.
- The key validity check is windowLength - maxFrequency <= k.
- Do not confuse substring with subsequence; the chosen block must be contiguous.

#08 LongestSubArrayWithOnesAfterReplacement

Medium

At-most-K zeroes window Medium Source LeetCode

Before reading: how many zeroes are currently spending flip coupons inside the window?

PROBLEM DESCRIPTION

Given a binary array and k flips, return the length of the longest contiguous subarray that can become all 1s by changing at most k zeroes to 1.

INPUT

`nums = [0, 1, 0, 0, 1, 1, 0, 1, 1, 0, 0, 1, 1], k = 3`

OUTPUT

`9`

TIME

O(n)

SPACE

O(1)

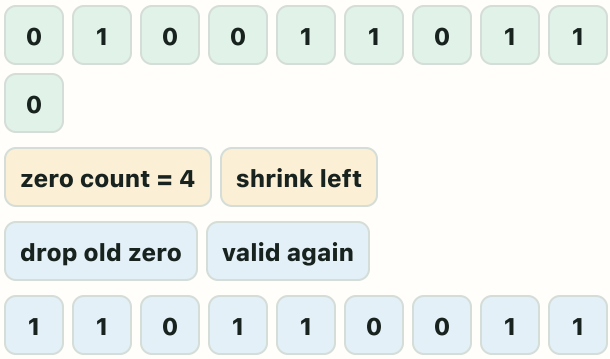
DATA STRUCTURES

Zero counter, left index, right index

WHEN TO RECOGNIZE IT

When the window is valid as long as it contains at most k zeroes.

VISUAL HOOK



1. Expand right and count a zero when it enters.
2. If zero count becomes more than k, move left until a zero leaves.
3. Once the window has at most k zeroes again, record its length.

Picture every zero as a flip coupon spent inside the window. The window may stretch while it uses at most k coupons; if it spends one too many, move left until one old zero leaves.

CORE MOVE

Expand right. Count zeroes inside the current window. While the zero count is greater than k, move left and decrease the count when a zero exits. Track the longest valid window.

CODE SKELETON

```
left = 0; best = 0; zeroes = 0
for right in 0..n-1:
  if nums[right] == 0: zeroes++
  while zeroes > k:
    if nums[left] == 0: zeroes--
    left++
  best = max(best, right - left + 1)
return best
```

BRUTE FORCE / BASELINE

Try every start index and extend right while counting zeroes. It is easy but can take O(n^2).

ALTERNATE APPROACHES

- This is the same shape as at-most-K distinct, but the tracked invalid item is only zero count.
- A prefix-sum plus binary search approach can find valid lengths, but the sliding window is simpler and O(n).

BUG MAGNETS

- The replacement count is the number of zeroes in the window, not the number of groups of zeroes.
- Shrink while zeroes > k, because the left side may pass over ones before a zero exits.
- Update the best length only after the window is valid.

#09 PermutationInString

Medium

Fixed anagram window Medium Source LeetCode

Before reading: has this fixed-size window paid every character debt from s1?

PROBLEM DESCRIPTION

Given strings s1 and s2, return true if any contiguous substring of s2 is a permutation of s1.

INPUT

`s1 = "dc", s2 = "ocdicf"`

OUTPUT

`true`

TIME

O(26 * n)

SPACE

O(1)

DATA STRUCTURES

int[26] frequency bank, fixed-size window, left index, right index

WHEN TO RECOGNIZE IT

When order does not matter, but the matching block must be contiguous and exactly the same length as s1.

VISUAL HOOK

d:1 **c:1**

window oc **not paid**

slide **undo o** **take d**

window cd **all debt paid**

1. Build a 26-count bank from s1.
2. Use a fixed window of length s1 inside s2.
3. Subtract the entering character and add back the leaving character; a balanced bank means permutation found.

Think of s1 as a character debt bank. A character entering the window pays one debt; a character leaving puts that debt back. A valid window leaves no unpaid positive debt.

CORE MOVE

Build the s1 frequency bank, then subtract the first s1-length window from s2. If the bank is balanced, return true. Otherwise slide one character at a time by adding back the left character and subtracting the new right character.

CODE SKELETON

```
if s1.length > s2.length: return false
bank = counts of s1
for right in 0..s2.length-1:
  bank[s2[right]]--
  if right >= s1.length:
    bank[s2[right - s1.length]]++
  if right >= s1.length - 1 and bank is all
zero:
    return true
return false
```

BRUTE FORCE / BASELINE

Generate or sort every length-s1 substring and compare it with s1. It is clear but costs extra work for every window.

ALTERNATE APPROACHES

- Use two int[26] arrays and compare them for each fixed-size window.
- Maintain a matched-count value to avoid scanning all 26 slots each time; the scan is still O(1), but matched-count is a nice optimization.

BUG MAGNETS

- The window size must stay exactly s1.length().
- Undo the outgoing left character before or while adding the incoming right character; do not let the window grow.
- Duplicate letters in s1 matter, so a Set is not enough.

#10 StringAnagrams

Medium

Find every anagram window Medium Source

Before reading: which fixed-size windows pay the full pattern debt, and what are their left indices?

PROBLEM DESCRIPTION

Given a string and a pattern, return every starting index where a contiguous substring is an anagram of the pattern.

INPUT

```
str = "cbaebabacdabc", pattern = "abc"
```

OUTPUT

```
[0, 6, 10]
```

TIME

O(26 * n)

SPACE

O(1)

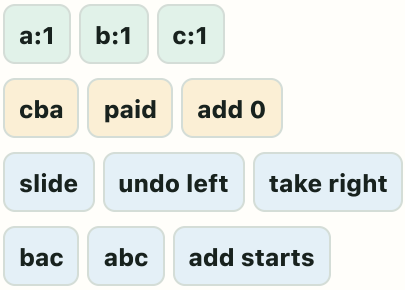
DATA STRUCTURES

int[26] frequency bank, result list, fixed-size window

WHEN TO RECOGNIZE IT

When you need all matching windows, not just whether one matching permutation exists.

VISUAL HOOK



1. Build the pattern frequency bank.
2. Slide a fixed window of pattern length across the string.
3. Whenever the bank is balanced, add the current left index to the answer list.

Reuse the same debt-bank idea as permutation-in-string, but instead of returning at the first match, collect the left index every time the fixed-size window pays all pattern debts.

CORE MOVE

Build the pattern count bank, subtract the first pattern-length window, and record index 0 if balanced. Then slide one step: add back the outgoing left char, subtract the incoming right char, and record the new left index whenever the bank is valid.

CODE SKELETON

```
answer = []
bank = counts of pattern
for right in 0..s.length-1:
  bank[s[right]]--
  if right >= pattern.length:
    bank[s[right - pattern.length]]++
  if right >= pattern.length - 1 and bank is
balanced:
    answer.add(right - pattern.length + 1)
return answer
```

BRUTE FORCE / BASELINE

Sort or count every pattern-length substring independently and compare with the pattern. It is clear, but repeats work for every window.

ALTERNATE APPROACHES

- Use two frequency arrays and compare all 26 counts for each window.
- Use a matched-count optimization to avoid scanning all 26 slots, though O(26) is still constant.

BUG MAGNETS

- Return starting indices, not the anagram strings.
- The window must stay exactly pattern.length().
- Duplicate letters in the pattern must be counted, so a Set cannot solve it.